Can a Large Language Model Win a Kaggle Competition?

Illya Gordyy[1], Estevan Rendon [2], Niful Islam[2], Mohammad Wardat[2,*], Huirong Fu[2]
[1]Department of Computer Science and Engineering, California State University, Sacramento, CA, USA
[2]Department of Computer Science and Engineering, Oakland University, Rochester, MI, USA
igordyy@csus.edu, {erendon, islam3, wardat, fu}@oakland.edu

***Abstract*—Large Language Models (LLMs) have demonstrated exceptional capabilities across diverse application domains. However, their potential in solving intricate problems within competitive environments, such as Kaggle competitions, remains relatively unexplored. This paper presents the first investigation into leveraging LLMs to autonomously build deep learning solutions specifically for Kaggle competitions. Our proposed method, *CompeteRAG*, employs a Retrieval-Augmented Generation (RAG) framework, utilizing top-voted Jupyter notebooks as exemplary templates to construct targeted deep learning solutions built with the Keras library and optimized using Keras Tuner. If initial code-generation attempts result in incomplete or erroneous outputs, *CompeteRAG* can analyze these errors and iteratively refine the generated code. We rigorously evaluate *CompeteRAG*'s efficacy using two state-of-the-art (SoTA) LLMs specialized in code generation: GPT-o4-mini and DeepSeek-R1. Additionally, we introduce a comprehensive dataset comprising highly-rated Jupyter notebooks that build DNN solutions for Kaggle competitions. Our study provides critical insights into the strengths and inherent limitations of current LLM methodologies when confronting complex structured-data tasks common in competitive environments.**

***Index Terms*—LLM, RAG, Keras, Keras Tuner, Deep Learning**

I. Introduction

Kaggle [1] is a popular online platform for data science and machine learning that provides access to datasets, coding notebooks, community discussions, and learning resources. Among its many features, Kaggle hosts competitions where individuals and teams tackle challenges by applying advanced analytics and machine learning techniques. These competitions represent real-world problems faced in healthcare [2], businesses [3], agriculture [4], and other domains. Deep neural networks (DNNs) have played a central role in many solutions [5]. Recently, Large Language Models (LLMs) have demonstrated significant performance in code generation [6], code repair [7], and code testing [8].

While prior research has tested LLMs' coding capabilities in isolated contexts [9], [10], only a few works have explored their ability to generate deep learning solutions from scratch in competitive environments like Kaggle. Since Kaggle competitions resemble real-world problems which takes significant time and resources for humans to solve, an LLM-driven end-to-end solution would demonstrate their practical applicability. However, LLMs' capabilities are limited to their training data, unless provided with additional information through mechanisms such as Retrieval-Augmented Generation (RAG). To the best of our knowledge, no prior work presents an end-to-end pipeline that incorporates architectures enhancing LLMs, such as RAG, to solve Kaggle competitions.

To fill this gap, we propose *CompeteRAG*, a retrieval-augmented framework that automates Kaggle competition solutions. Our method uses similarity search on curated high-quality notebooks to find the closest match, which then guides the LLM in generating a complete solution. If the first attempt fails, a feedback loop analyzes the error log and prior code to produce a fixed version. We test the approach on generating Keras (TensorFlow) and Keras Tuner solutions for both classification and regression tasks, against multiple baselines. Experiments show average classification scores of 0.69 (Keras) and 0.72 (Keras Tuner) using GPT-o4-mini, compared to 0.81 for the best human solution. Moreover, our method required far fewer attempts—1.3 (Keras) and 1.7 (Keras Tuner) compared to 44.5 for humans. In summary, our major contributions are as follows:

- We present *CompeteRAG*, the first RAG-based solution for solving Kaggle competitions through deep learning approaches.
- We carefully design the architecture of *CompeteRAG* to extract the closest DNN solution as an example for the LLM, with an iterative feedback loop to refine incomplete code generations.
- We have conducted an extensive study of the proposed solution using two state-of-the-art language models, GPT-o4-mini and DeepSeek-R1, and the results demonstrate competitive performance compared to humans.
- We present a dataset containing 504 Kaggle notebooks with deep learning solutions to Kaggle competitions, which can be leveraged for future research.

The remainder of this paper is organized as follows. Section II details the motivation for this work. In Section III, we present the full architecture of our proposed approach followed by the results obtained from the approach in Section IV. Subsequently a summarization of the related research works are presented in Section V. Finally, we discuss threats to validity in Section VI and conclude with our findings and directions for future work in Section VII.

* Corresponding author: Mohammad Wardat, wardat@oakland.edu

II. Motivating Example

The process of developing high-performing models for complex tabular data problems, which are common on platforms like Kaggle, presents a significant challenge. Success in these competitions typically hinges on a combination of sophisticated feature engineering, algorithm selection, and meticulous hyperparameter tuning [11]. This workflow is fundamentally iterative and knowledge-intensive, demanding deep expertise and a time-consuming, trial-and-error approach. For a human expert, achieving a top-tier result often requires dozens of submissions to manually discover the optimal strategy, making the process prohibitively slow and resource-intensive.

The Kaggle Tabular Playground Series (May 2022) [12] is an example of a binary classification challenge in Kaggle focused on predicting a simulated manufacturing machine's state as either 0 or 1. The dataset has 32 features including both categorical and continuous variables, one class column and a total of 900,000 instances in the training and 700,000 instances in the test dataset. The complexity of the dataset makes it challenging to solve. As of August 2025, the best-performing team in the competition has made 80 attempts to reach a near-perfect score, while the top 10 teams have an average of 50.7 attempts. While the number of attempts reflects the iteration effort, it does not account for the substantial time spent on problem analysis, extensive exploratory data analysis (EDA), feature engineering, brainstorming, failed experiments, and other behind-the-scenes work.

TABLE I: Performance Comparison on the Tabular Playground Series (May 2022)

Model	**Approach**	**Public Score (AUC)**	**Training Time (s)**	**Attempts**
GPT-o4-mini	Zero-Shot	0.79	516.00	2
	RAG + Keras	0.86	607.29	1
	RAG + Keras Tuner	0.90	4625.43	1
DeepSeek-R1	Zero-Shot	0.82	160.80	1
	RAG + Keras	0.95	56.21	1
	RAG + Keras Tuner	0.96	12421.66	2
Benchmark	**Human Expert**	**1.00**	N/A	**80**

Therefore, an automated approach can reduce human effort and accelerate the path to optimal solutions. While recent LLMs are expert in generating code [13], their performance can be enhanced by integrating examples in the prompt through Retrieval-Augmented Generation (RAG) based systems [14]. To resolve this dilemma, we propose *CompeteRAG*, a RAG-based technique that augments an LLM's generative process with information retrieved from an external knowledge base. Specifically, we equip the LLM with a curated repository of high-performing code and strategies from past data science competitions. This allows the model to reason based on proven, contextually relevant examples, effectively injecting expert-level intuition into its problem-solving process.

In the Tabular Playground Series (May 2022) competition, a human expert required more than 80 submissions to achieve a perfect 1.00 AUC score, whereas a baseline LLM quickly produced a non-competitive score of 0.82 AUC, as shown in Table I. In contrast, *CompeteRAG* achieved a near-expert AUC of 0.96 in just two attempts. This result establishes an optimal trade-off, delivering elite performance without the prohibitive time cost and confirming our approach as a powerful method for unifying expert strategy with AI efficiency.

III. Approach

In this section, we present the data collection approach for the study along with the proposed approach in detail. Section III-A outlines our data collection process, and in Section III-B, we introduce *CompeteRAG*, our retrieval-augmented generation (RAG) framework.

A. Data Collection and Preprocessing

1) Initial Competition Collection and Split: We first constructed a dataset comprising Kaggle Playground competitions containing exclusively textual or tabular data [15]. The dataset was specifically curated to exclude competitions involving image data to ensure consistency and relevance for text and tabular problem-solving. Using the Kaggle public API [16], we listed a full collection of all the Playground competitions available in the section. However, due to the limitations in both Kaggle filtering and the existing API, it required manual filtering to ensure that all of the possible datasets are included and none of them include heavy files, specifically 10 GB and above. Additionally, due to Kaggle's competition data-download and solution-submission policy, it required manually joining the competitions first, before receiving access to the data. Finally, we split the existing collected competitions into training and testing competitions, with approximately 75% of competitions being in our knowledge base, or training dataset, and 25% composing our testing dataset. To be more specific, 64 competitions were delegated for our knowledge base, and the other 20 were used for our testing procedures. Although the data splitting was performed randomly, we ensured that the testing set contained a comparable number of classification and regression problems to enable a more balanced and fair evaluation.

Along with the dataset, we also collected Jupyter notebooks that present deep learning solutions related to the competition. To select the notebooks, we searched the competition's code section separately using the keywords TensorFlow and PyTorch. From each search, we selected up to five of the highest-voted notebooks. If fewer than five notebooks were available for a search, we included all that were present.

2) Preprocessing: Since both the notebooks and the Kaggle competition descriptions contain large amounts of text, including unnecessary information that could mislead the LLM and consume the context window, we performed preprocessing on the selected dataset. In this stage, we extracted specific information such as the problem type (e.g., classification or regression), problem subtype (e.g., binary classification, multi-class classification), a summarized problem description, dataset type (e.g., tabular, time-series), class column, and the evaluation metrics used. For the notebooks, we extracted the most critical preprocessing steps and the model generation

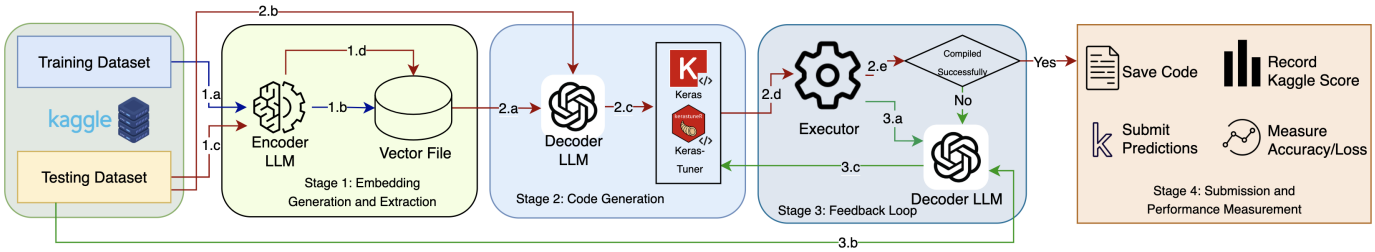

Fig. 1: Overview of the *CompeteRAG* approach.

components to ensure all relevant information fits within the context window. For this preprocessing task, we employed the structured output method of the corresponding LLMs (i.e., GPT-4o-mini and DeepSeek-R1).

B. Proposed CompeteRAG

For evaluating the completeness of RAG in Kaggle competitions, we developed *CompeteRAG*. An overview of the proposed approach is presented in Figure 1. As presented in the diagram, the proposed approach consists of four major steps. The following sections present the first three stages in detail.

1) Embedding Generation and Extraction: This is the first stage of the proposed solution. The goal of this stage is to encode and store the training dataset, and then retrieve the top-k most similar examples for a given test sample to provide as input examples to the LLM in the next stage. In this stage, we merged competition descriptions with dataset metadata, collected in Section III-A2, into a unified textual representation. This block was then fed to a RoBERTa-base transformer fine-tuned with the DiffCSE contrastive learning framework [17], producing unit-normalized dense sentence embeddings (1.a). We adopted this encoder as DiffCSE markedly improves the semantic quality of RoBERTa embeddings without requiring extra labeled data. This makes it well-suited for matching competitions and datasets by meaning rather than simple keyword overlap. Concurrently, each categorical attribute in the metadata field was converted into a weighted one-hot vector to reflect its relative importance. For example, the problem subtype was given the highest priority with a weight of 3, followed by the problem type (2.5), evaluation metrics (2), and finally the dataset type, competition description, and dataset description, each with a weight of 1. These weights were assigned based on attribute priority, though further investigation is still needed.

After assigning weights to the categorical vectors, they were concatenated with text embeddings and L2-normalized to reduce noise and outliers. This integration allowed the resulting embeddings to capture both the semantic context of competition descriptions and the structural context of competition types. The vectors were then stored as a pickle file (1.b) to avoid redundant operations. Finally, we constructed an inner-product–based FAISS [18] index over these normalized vectors to enable fast, semantically driven retrieval. The testing dataset also passes through the same encoding process (1.c). For each test sample, a normalized query vector was created by encoding its text and categorical metadata, and a similarity search was performed against the prebuilt FAISS index (1.d) to retrieve the top k most similar entries and return it for the second stage (2.a). For similarity search, we used the Flat Index with Inner Product (FlatIP) method [18], which is computationally intensive but provides exact results. However, since our training dataset is relatively small, the higher resource usage did not significantly impact performance. In addition, due to the limited context window of the LLM, we used a k value of 1 throughout the experiment.

2) Code Generation: This stage is responsible for generating a complete solution for the test sample. An LLM is prompted to generate full code by referencing an example retrieved in the previous step (2.a) and incorporating insights from the test dataset (2.b), including relevant statistics. Given a well-structured prompt, the LLM produces Keras (TensorFlow) or Keras Tuner code. While generating the Keras code is a one step process, generating Keras Tuner code requires an additional step after generating the original Keras code as explained in Section III-C. The prompt for generating the Keras and Keras Tuner code is presented in Section III-D. Once generated, the code is executed (2.d); if it runs successfully on the first attempt, its predictions are submitted, and the resulting scores and performance metrics are recorded. If the code fails to execute correctly, the process advances to the third stage.

3) Feedback Loop: This section applies only if the decoder LLM fails to generate a complete executable code on the first attempt in stage 2. The task of this section is to get the previous code along with the error log (3.a) and the test dataset description similar to the previous stage (3.b) to generate the fixed code (3.c) that goes through the execution and validation process, illustrated in stage 2 (2.d and 2.e). If it fails to generate complete code after 4 attempts, the experiment is considered a failure. However, according to our experiment, the model typically fixes a code within one or two iterations.

C. Keras Tuner Code Generation

Before integrating the tuner, we built a detailed set of hyperparameter options based on prior research and the Keras Tuner guidelines. This included setups for multilayer perceptrons, tabular classifiers, time-series LSTM models, and regression models. For each model type, we defined the ranges, types, and sampling methods of the hyperparameters, drawing on findings

TABLE II: Hyper-parameter Search Ranges by Model Subtype

Model Subtype	Layers	Units	Batch	Epochs	Dropout
Low-feature tabular MLP	2–8	64–1024	32–1024	20–200	0–0.5
High-feature classifier	2–8	64–512	64–512	10–50	0–0.5
LSTM time-series	1–4	LSTM: 32–256 Dense: 16–128	32–256	10–100	0–0.5

from studies in deep-learning optimization [11], [19]–[21]. Previous work has shown that Bayesian optimization, in particular, often provides more efficient searches and better results [22]–[24]. Therefore, we selected Bayesian optimization as our search strategy. These model-specific profiles act as starting points for the tuner, guiding it to explore the most promising regions of the hyperparameter space. Table II summarizes the chosen ranges for each model subtype.

D. Prompt Construction and Utilization

Generate a runnable deep learning model using Keras in Python code. The generated code must implement:

- **Imports:** Load core libraries for tabular ML and Keras.
- **Data Handling:** Auto-load files, split train/validation, encode target per task subtype.
- **Pre-processing:** Impute missing values, cap categorical levels, scale numerics, build unified transformer.
- **Model Architecture & Compilation:** Choose 2–4 hidden layers with ReLU; compile with Adam; set output activation/loss by task (sigmoid + BCE for binary, softmax + CCE for multiclass, linear + MSE for regression).
- **Training:** Use early stopping and model checkpointing.
- **Prediction & Submission:** Generate predictions and save competition-ready CSV.

Fig. 2: Core Keras-notebook prompt used by *CompeteRAG*.

To generate the Keras Tuner code, the LLM takes the previously defined Keras model definition block and the set of hyperparameters for the given model subtype. It then generates a tuner-compatible version of the model, which is later merged with the unchanged parts of the pipeline such as data loading, preprocessing, callbacks, and submission code. The two-step process of generating the Keras Tuner code resulted in fewer errors compared to asking the LLM to produce the entire code from scratch.

A summarized form of the prompt used to generate the Keras code is presented in Figure 2. The prompt explicitly guided the LLM to generate runnable Python code using Keras. This structured prompt included detailed instructions for setting reproducibility seeds, necessary library imports (such as pandas, NumPy, scikit-learn, and TensorFlow), data handling procedures including train validation splits, target encoding mechanisms based on the problem subtype (binary classification, multiclass classification, multi-label classification, regression, and time-series forecasting), feature engineering guidelines, and preprocessing pipeline construction (using imputation, scaling, and one-hot encoding). Furthermore, the prompt provided comprehensive guidelines on model architecture decisions (such as layer depth, unit sizes, and activation functions), compilation settings tailored to specific problem subtypes (including appropriate loss functions and evaluation metrics), and training routines incorporating early stopping and model checkpoint callbacks. Finally, the generated code was instructed to perform model evaluation, logging relevant metrics into a JSON file, and producing competition-compliant prediction submissions.

Return a runnable Python code wrapped in <Code>..</Code>

- **Reuse of Keras Code:** Keep existing model and callbacks unchanged.
- **Profile Selection:** Pick best hyperparameter schema by tag overlap.
- **Callback Preservation:** Retain original early stopping and checkpoints for search and training.
- **HyperModel Definition:** Define custom HyperModel with spaces from chosen profile.
- **Bayesian Optimization:** Perform search via Bayesian tuner with given batch size and epoch ranges.
- **Search Execution:** Run `tuner.search` with validation and callbacks.
- **Rebuild and Retrain:** Apply best hyperparameters, rebuild, and retrain with original setup.

Fig. 3: Keras Tuner generation system prompt.

Similarly, to generate a Keras Tuner code, we have leveraged the prompt presented in Figure 3. The summarized prompt illustrates the incorporated Keras Tuner functionalities. This phase's prompt instructed the LLM to retain the existing model definition and callback configurations from the previously generated Keras notebook. It guided the selection of the most suitable hyperparameter profile from a predefined hyperparameter bank based on metadata similarity. The prompt specified creating a custom HyperModel class for explicitly defining the hyperparameter search space (including the number of layers, units, activation functions, dropout rates, optimizers, and learning rates). Bayesian optimization was strictly enforced for conducting the hyperparameter search, ensuring the efficient exploration of the hyperparameter space. Post-optimization, the prompt ensured rebuilding and retraining the model with the best-found hyperparameters, preserving the original callback mechanisms and validation strategies. While both prompts shown here are summarized versions of the originals, the complete prompts are provided in [25].

E. Proposed CompeteRAG Algorithm

The algorithm presented in this study involves three core stages designed to automate and enhance deep learning so-

lutions for Kaggle competitions.

Algorithm 1 BuildKnowledgeBase

```
1: Build Knowledge Base
   Input: all_competitions
   Output: FAISS index, testSet
2: testSet ← 10 random classification ∪ 10 random regression from all_competitions
3: trainSet ← all_competitions − testSet
4: for all comp in trainSet do
5:     meta ← LLM_structured_output(comp)
6:     notebooks ← DownloadBestNotebooks(comp, 5, TF) ∪ DownloadBestNotebooks(comp, 5, PT)
7:     for all nb in notebooks do
8:         preprocessingSteps ← ExtractSteps(nb)
9:         modelCode ← ExtractModel(nb)
10:        Save(preprocessingSteps, modelCode)
11:    end for
12:    catVec ← EncodeCategorical(meta, weights)
13:    textMeta ← meta.problemDesc || meta.datasetDesc
14:    txtVec ← TextEmbed(textMeta)
15:    vec ← NormalizeL2([catVec, txtVec])
16:    FAISS.Add(vec)
17: end for
    return FAISS index, testSet
```

Algorithm 1 illustrates the first stage of *CompeteRAG*. It takes all the selected Playground competitions ($all_competitions$) as input and returns the constructed FAISS index along with the test set. The algorithm starts by randomly selecting 10 classification and 10 regression problems for the $testSet$ (Line 2), while the remaining competitions are assigned to the $trainSet$ (Line 3).

Subsequently, we extract metadata such as problem types, subtypes, evaluation metrics, and dataset descriptions from each competition in the *trainSet* using a custom parser combined with LLM-based classification (Line 5). Additionally, details of high-performing solutions written in TensorFlow and PyTorch, including preprocessing steps and model architectures, are also recorded (Line 7 -11). The metadata collected in Line 5 includes categorical features such as problem type, problem subtype, evaluation metric, and dataset type. These features are encoded and weighted using the previously defined weights and saved in *catVec* (Line 12). Meanwhile, the textual metadata (*textMeta*) is stored separately and encoded with a RoBERTa transformer (Line 12). Finally, the categorical and textual vectors are combined and normalized (Line 15) before being added to the FAISS index.

The second and third stages of the proposed *CompeteRAG* are presented in Algorithm 2. This algorithm automates the generation of executable deep learning solutions using Keras and attempts to fix any errors if the first generation fails. It takes the *testSet* produced in the previous stage, along with the value of k (the number of example samples provided to the LLM) and $ktCode$, a binary variable indicating whether to generate Keras or Keras Tuner code, and returns a list ($generatedCodes$) containing either a complete solution or a failure message if code generation is unsuccessful. The algorithm begins by initializing an empty list, $generatedCodes$, to store the final code, and iterates through all test samples. For each test sample, it collects the metadata and downloads the corresponding training, testing, and sample submission files. Subsequently, it prepares the embedding vector for the test sample following the steps outlined in Algorithm 1 (Lines 4–8). Finally, it searches the FAISS index to retrieve similar notebooks (Lines 9–16). For similar notebooks, we only collect the preproceesing and model code, which is later used to construct the prompt (Line 18). In the next step, a complete code is generated by passing the prompt into an LLM (Line 21). In scenarios when $ktCode$ is set to true, indicating that Keras Tuner code should be generated, the $generatedCodes$ is modified by replacing specific parts of the previous code with Keras Tuner–compatible code (Lines 23-25).

If the code executes successfully, it is saved into $generatedCodes$. Otherwise, a fix prompt is created using the previous code, the execution log, and the data schema to correct the code. A maximum of four attempts are allowed to fix the code. If a complete code is not generated within these attempts, the experiment is considered a failure (Lines 34–36). Once code has been generated for all test instances, $generatedCodes$ is returned as the output of the algorithm.

Algorithm 2 GenerateSolution

```
   Input: testSet, k, ktCode
   Output: Generated code pipelines for each competition in testSet
1: generatedCodes ← ∅
2: for all newComp in testSet do
3:     newMeta ← CollectMetadata(newComp)
4:     schemaFiles ← Download([trainingFiles, testingFiles, sampleSubmission])
5:     catVec ← EncodeCategorical(newMeta, weights)
6:     textMeta ← newMeta.problemDesc || newMeta.datasetDesc
7:     txtVec ← TextEmbed(textMeta)
8:     newVec ← NormalizeL2(catVec || txtVec)
9:     neighborIDs ← FAISS.Search(newVec, k)
10:    similarNotebooks ← {}
11:    for all id in neighborIDs do
12:        payload ← FAISS.Payload(id)
13:        similarNotebooks ← similarNotebooks ∪ payload.solutions
14:    end for
15:    preprocessingData ← Aggregate(similarNotebooks.preprocessing)
16:    codeData ← Aggregate(similarNotebooks.modelCode)
17:    sData ← Inspect(schemaFiles)
18:    prompt ← ComposePrompt(sData, preprocessingData, codeData)
19:    attempts ← 0
20:    while attempts < 4 do
21:        generatedCode ← LLM(prompt)
22:        attempts ← attempts + 1
23:        if attempts = 1 and ktCode = True then
24:            generatedCode ← convert_to_KT(generatedCode)
25:        end if
26:        log = Execute(generatedCode)
27:        if log has no error then
28:            generatedCodes ← generatedCodes ∪ generatedCode
29:            break
30:        else
31:            prompt ← ComposeFixPrompt(generatedCode, log, sData)
32:        end if
33:    end while
34:    if attempts = 4 then
35:        generatedCodes ← generatedCodes ∪ failed
36:    end if
37: end for
    return generatedCodes
```

IV. Evaluation

This section presents the comprehensive evaluation of our proposed *CompeteRAG* approach. We begin by detailing the research questions, experimental setup, the dataset used, and the baselines for comparison. We then present our results, first with a comprehensive performance overview, before addressing our research questions individually, presenting the relevant results and analysis. Finally, we discuss the broader implications of our findings.

A. Research Questions

The research is conducted to answer the following research questions:

- **RQ1:** How effective is the proposed *CompeteRAG* approach in solving Kaggle competitions?
- **RQ2:** Which language model, GPT-o4-mini or Deepseek-R1, demonstrates superior performance in Kaggle competitions?
- **RQ3:** How does the performance of our approach compare to baseline scores?

B. Experimental Setup

All experiments were conducted on a Jetstream2 instance featuring a 40GB A100 GPU, accessed via SSH within Visual Studio Code. Our implementation uses Python 3.12.3 in a virtual environment and leverages standard Python libraries, including Pandas, NumPy, Transformers, Scikit-learn, TensorFlow, FAISS, along with the Kaggle, OpenAI and DeepSeek APIs for downloading Kaggle datasets and accessing LLMs.

C. Dataset

Our evaluation uses a curated set of 20 Kaggle Playground competitions, evenly split into 10 regression and 10 classification tasks, to enable robust analysis across these supervised learning paradigms. The competitions include datasets of varying sizes and complexities to test our methods on diverse challenges. While the test set is balanced, the training set contains 40 classification and 24 regression tasks, reflecting the smaller number of regression competitions in Kaggle Playground. To provide relevant, high-quality examples, we constructed a knowledge base from 504 high-scoring public notebooks, from which each model variation performs retrieval during task execution.

D. Baselines

To effectively contextualize the performance of our *CompeteRAG* approach, we compare it against two distinct baselines.

1) Zero-Shot: In this configuration, the language model generates a solution based solely on the competition overview and its internal knowledge, without access to any retrieved documents. Compared to our proposed approach, the zero-shot method skips Stage 1 (i.e., embedding generation and extraction). Stage 2 is slightly modified to exclude the example extracted from stage 1 (2.a) from the prompt. The remainder of the approach follows *CompeteRAG*. This baseline represents the foundational capability of the LLM.

2) Human: This baseline is established by the top-performing public score for each Kaggle competition, as recorded in July 2025. This allows us to benchmark our automated approach against the pinnacle of human competitive performance.

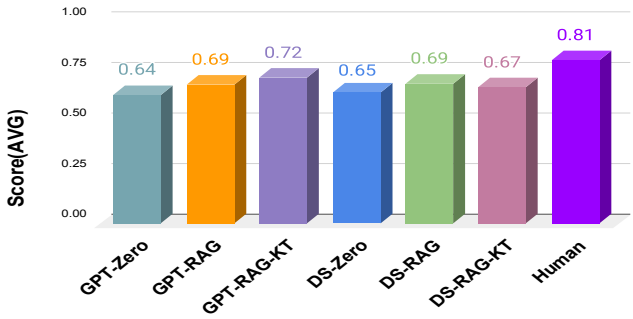

Fig. 4: Classification Score: LLM vs Human.

E. Results and Analysis

Having detailed the benchmark construction, we now present the evaluation results, beginning with the comprehensive performance overview in Table III. The table illustrates the average performance of the LLMs in different settings along with the top human score. For simplicity, we use GPT-Zero to denote GPT-o4-mini in the zero-shot setting, and DS-Zero to denote DeepSeek-R1 in the zero-shot setting. Similarly, GPT-RAG refers to our method equipped with GPT-o4-mini for generating Keras code, while GPT-RAG-KT indicates the addition of Keras Tuner. The same naming convention is applied to DeepSeek-R1, resulting in DS-RAG and DS-RAG-KT.

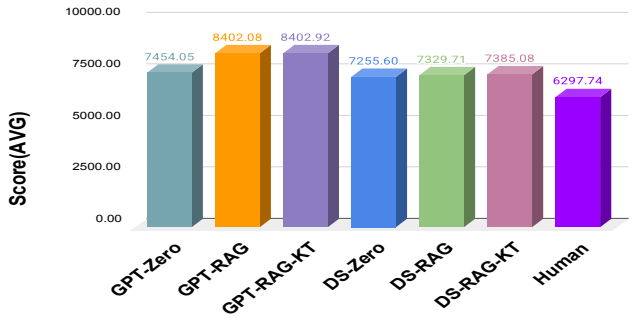

Fig. 5: Regression Score: LLM vs Human

1) RQ1: How effective is the proposed CompeteRAG approach in solving Kaggle competition tasks?: The effectiveness of *CompeteRAG* is measured by its impact on three key metrics: final competition score, training time (computational cost), and the number of attempts required to generate valid code (robustness).

As shown in Figure 4, *CompeteRAG*: Keras significantly improves final scores in classification tasks for both GPT-o4-mini (+5%) and DeepSeek-R1 (+4%) over the Zero-Shot baseline, with the *CompeteRAG*: Keras Tuner pushing the GPT-o4-mini score even higher to a final 0.72 (+8% over baseline).

TABLE III: Comparison of model performance across all evaluated metrics.

LLM VS Human	Strategy	Task Type	Public Score	Private Score	Training Accuracy	Validation Accuracy	Traininig RMSE	Validation RMSE	Training Time	Attempts
GPT o4-mini	Zero-Shot	Classification	0.64	0.64	0.74	0.71	-	-	168.41	1.90
		Regression	7454.05	20971.71	-	-	8234.97	7439.54	159.01	1.80
	RAG + Keras	Classification	0.69	0.69	0.79	0.80	-	-	263.17	1.30
		Regression	8402.08	177869.36	-	-	6135.82	5889.84	418.28	1.50
	RAG + Keras Tuner	Classification	0.72	0.72	0.84	0.84	-	-	1493.59	1.70
		Regression	8402.92	7676.68	-	-	5291.15	5129.31	1102.42	1.50
Deepseek-R1	Zero-Shot	Classification	0.65	0.65	0.81	0.77	-	-	194.61	1.40
		Regression	7255.60	23683.71	-	-	7057.06	6289.62	347.03	1.60
	RAG + Keras	Classification	0.69	0.70	0.76	0.76	-	-	44.49	2.10
		Regression	7329.71	17141.61	-	-	7478.92	7621.31	326.52	1.70
	RAG + Keras Tuner	Classification	0.67	0.68	0.76	0.74	-	-	1458.58	1.90
		Regression	7385.08	18175.80	-	-	7611.06	7133.71	656.81	1.70
Human		Classification	0.81	-	-	-	-	-	-	44.5
		Regression	6297.74	-	-	-	-	-	-	64

However, for DeepSeek-R1, the optimized model performed worse than its standard configured counterpart, with its score decreasing from 0.69 to 0.67. However, its effectiveness in regression was mixed, as presented in Figure 5, decreasing GPT-o4-mini's score by [12.7%] (from 7454 to 8402, where lower is better). The *CompeteRAG*: Keras Tuner optimization offered no solution here, as it yielded a nearly identical score of 8402.92. In terms of efficiency, presented in Figure 6 , CompeteRAG: Keras substantially increased training time for GPT-o4-mini (+56%). In contrast, the models found by *CompeteRAG*: Keras Tuner proved to be far more computationally expensive, increasing the average training time dramatically in all scenarios, in some cases by over 3000% (e.g., for DeepSeek-R1 classification, from 44.5s to 1458.6s). Finally, as detailed in Table IV, the approach improved robustness for GPT-o4-mini, cutting attempts by over 30%, but had the opposite effect on DeepSeek-R1, increasing its attempts by 50% for classification. The *CompeteRAG*: Keras Tuner models presented a mixed trade-off here: for GPT-o4-mini, the tuner partially reversed the robustness gains (from 1.30 to 1.70 attempts), while for DeepSeek-R1, it mitigated some of the harm, reducing attempts from 2.10 to 1.90.

TABLE IV: Average Number of LLM Attempts.

Approach	**Classification Attempt (AVG)**	**Regression Attempt (AVG)**
GPT-Zero	1.90	1.80
GPT-RAG	1.30	1.22
GPT-RAG-KT	1.70	1.50
DS-Zero	1.40	1.60
DS-RAG	2.10	1.60
DS-RAG-KT	1.90	1.70
Human	44.50	64.00

RQ1 Summary: The *CompeteRAG* approach achieves a lower classification and regression performance compared to the best human score but takes significantly less number of attempts to reach there.

2) RQ2: Which language model, GPT-o4-mini or Deepseek-R1, demonstrates superior performance in Kaggle competition tasks?: To determine the superior language model, we conducted a head-to-head comparison across all experimental conditions. In terms of raw score, the models were highly competitive in Keras configurations, with DeepSeek-R1 holding a slight edge in Zero-Shot classification (0.65 vs. 0.64) and *CompeteRAG*: Keras regression, while the two models performed identically in *CompeteRAG*: Keras classification (0.69). However, automated tuning with *CompeteRAG*: Keras Tuner revealed a key divergence in classification, as shown in Figure 4: the optimized GPT-o4-mini model achieved a score of 0.72, significantly outperforming DeepSeek-R1's 0.67. Conversely, DeepSeek-R1 maintained a consistent and substantial advantage in all regression scenarios in Figure 5. The most significant differentiator in the Keras configurations was efficiency. As illustrated in Figure 6, DeepSeek-R1 consistently required less training time. This efficiency gap, however, narrowed dramatically under automated tuning, which drastically increased training time for both models in classification (to 1460-1490s). Finally, a critical differentiator was robustness (Table IV); in nearly all *CompeteRAG* configurations, GPT-o4-mini proved more reliable, requiring fewer code generation attempts than DeepSeek-R1 (e.g., 1.30 vs 2.10 in *CompeteRAG* classification).

RQ2 Summary: Neither model is universally superior, as the choice is task-dependent: DeepSeek-R1 excels in regression and baseline efficiency, while GPT-o4-mini achieves a higher peak classification score through optimization and demonstrates consistently greater robustness.

3) RQ3: How does the performance of our approach compare to baseline scores?: This question is addressed by comparing *CompeteRAG* against the Zero-Shot and Human baselines. As shared in Figure 4, *CompeteRAG* consistently and significantly outperforms the Zero-Shot baseline in classification tasks, confirming the value of providing external knowledge for this task type. Conversely, in our regression experiments (Figure 5), the approach did not improve upon the Zero-Shot scores. When compared to the top human competitors (e.g., a classification score of 0.81), our approach closes the gap considerably. For instance, with GPT-o4-mini, the *CompeteRAG*: Keras configuration closed 29.4% of the

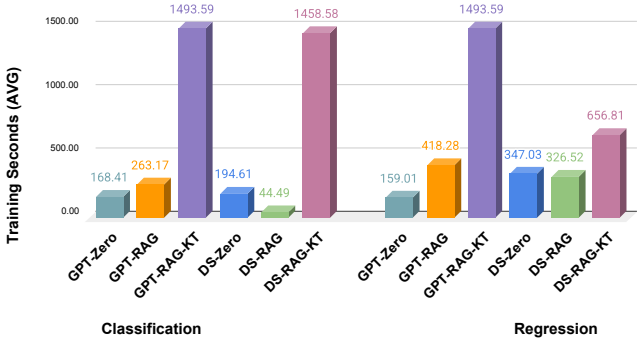

Fig. 6: LLM Training Time - Average in Seconds

performance gap to this human baseline, and the Keras Tuner optimized model pushed this further, closing 47.06% of the gap with a final score of 0.72. The key difference lies in the process: our automated approach achieves these near-human scores in a single, but long, computational run, whereas top human performance often requires numerous manual iterations and extensive domain expertise over days or weeks.

RQ3 Summary: For classification tasks, the *CompeteRAG* approach significantly outperforms the Zero-Shot baseline to deliver near-human results automatically, but it is not effective for regression, where it underperforms the baseline.

F. Discussion

Our findings indicate that the *CompeteRAG* framework is a significant step toward automating competitive data science, though its benefits are nuanced and model-dependent. Automated tuning can improve models like GPT-o4-mini to a higher classification score (0.72), while the same process may reduce performance for models like DeepSeek-R1. Although *CompeteRAG* did not surpass the best human scores, it outperforms average human performance and requires far fewer attempts, making it a strong starting point for further improvements. Additionally, among all the experiments, only two of them failed to generate a complete solution after 4 attempts.

The framework's higher performance over zero-shot methods in classification tasks shows that RAG effectively retrieves relevant examples to boost results. Its lower performance on regression tasks reflects a failure to extract suitable examples, likely due to the limited number of regression problems in the training set. Even though the test set balanced classification and regression tasks, the scarcity of high-quality regression notebooks in Kaggle Playground restricted the framework's access to suitable samples, a problem that can be addressed by collecting more regression-related notebooks.

Another limitation is the lack of hyperparameter tuning, which can significantly affect LLM performance (e.g., temperature, top-p). Future studies can explore the impact of these parameters. Overall, *CompeteRAG* provides a RAG-based approach that improves zero-shot LLM performance and offers a promising direction for automating the solution of Kaggle competitions.

V. Related Work

A. AutoML

AutoML aims to automatically configure predictive system architectures, reducing manual effort and specialized expertise. Popular techniques include Auto-Sklearn 2.0 [26] (built on Scikit-learn), Auto-WEKA [27] (Weka), and Auto-Keras [28] (Keras). Recent work has explored multilayer stacking strategies, e.g., AutoGluon [29]. A major limitation of these frameworks is their reliance on neural architecture search (NAS) [30], which is resource- and time-intensive. To address this, Manas [31] selects initial models from a GitHub-derived repository, assuming models for similar tasks provide better starting points. However, most approaches remain rule-based and restricted to predefined options. In contrast, LLM-based methods such as AutoML-Agent [32] and AutoM3L [33] are more efficient, leveraging training on large, diverse open-source datasets.

B. LLM for Software Engineering

Within software engineering, large language models (LLMs) leverage their understanding of natural language and code to assist across development, debugging, testing, and documentation—processes that are typically time-consuming. Recent models like Codex [34], StarCoder [35], CodeGen [36], and CodeT5 [37] have shown strong proficiency in code generation. Researchers have applied LLMs to tasks such as code generation [10], [38], [39], automatic program repair [40]–[43], design issue localization [9], testing [44], [45], and more. Integrating LLMs into software development has improved efficiency by reducing time, cost, and errors. However, they can still make mistakes, particularly in emerging domains outside their training data, where retrieval-augmented generation (RAG) solutions are especially valuable. The next section discusses how RAG architectures address these challenges.

C. Retrieval-Augmented Code Generation

Retrieval-augmented code generation enhances LLM-based code generation by extending prompts with relevant external knowledge. Unlike traditional LLMs limited to learned knowledge, RAG integrates external sources, making them more capable for specific tasks. As it requires no fine-tuning and scales easily, many researchers use it for code generation solutions. RepoCoder [46] retrieves relevant code snippets from unfinished code in the same repository for code completion. GameUnitGen [47] uses RAG to extract class and method metadata, builds context-rich prompts, and generates unit tests with a fine-tuned LLM. RA-DIT [48] employs RAG with dual training to jointly optimize a retriever and generator by aligning function-level queries with relevant snippets via contrastive learning. SOSecure [49] leverages RAG to retrieve Stack Overflow discussions with known vulnerabilities and antipatterns to prompt secure code revision, without retraining. DLCodeGen [50] introduced a two-stream RAG framework in

which one stream retrieves semantically similar code samples from a task-specific pool, while the other abstracts them into high-level templates, and the two are later combined to guide code generation.

Despite advances in AutoML and RAG-based code generation, their use for deep learning code generation—especially in real-world scenarios like Kaggle competitions—remains underexplored. We therefore present *CompeteRAG*, a RAG-based solution for developing models capable of participating in Kaggle competitions.

VI. Threats to Validity

A. Internal Threat

Our approach is tested on a single framework (i.e., TensorFlow/Keras), which may limit its generalizability to others such as PyTorch. To mitigate this threat, we collected notebooks from both PyTorch and TensorFlow, which were later used as examples for the LLM. The LLM's ability to generate correct Keras code even when provided with PyTorch examples demonstrates its robustness across frameworks. The future work will include studies in PyTorch and other major frameworks. Since LLMs generally vary their output across runs, a major threat is the inability to reproduce the same Keras model in every call. To mitigate this variability, we set both the temperature and top-p values to 1 throughout the experiments. Lastly, despite explicitly specifying the output format in the prompt, the LLM may not always adhere to it, making the architecture volatile. To address this, we employed structured output via function calling to ensure that the LLM consistently returns results in a predetermined format.

B. External Threat

The main external threat arises from the evaluation dataset. Since the competitions are publicly available, it is possible that the LLM was exposed to them during training. Nevertheless, this is unlikely to have significantly influenced the results, as integrating external examples through the RAG approach led to substantial fluctuations in performance. Future studies can mitigate this threat by evaluating the approach on newly released or restricted competitions. Additionally, the dataset was collected in July 2025, and any subsequent additions, modifications, or deletions of instances may affect the verifiability of this work.

VII. Conclusions and Future Work

This research bridges the gap between the slow, iterative process of human experts and the faster but often weaker performance of standard AI in competitive data science by introducing *CompeteRAG*, a framework that augments Large Language Models with a knowledge base of winning strategies. Our analysis shows that *CompeteRAG* is highly effective for classification, consistently outperforming Zero-Shot baselines, though less effective for regression. Although our comparison of two state-of-the-art LLMs revealed no universal winner, DeepSeek-R1 excelled in regression and efficiency, while GPT-o4-mini achieved higher classification scores and greater robustness. Against external baselines,*CompeteRAG* significantly outperforms Zero-Shot methods in classification and narrows the gap to top human performance by up to 47%. The main contribution of our research lies in presenting near-human solutions by trading intensive manual effort for substantial computational cost. *CompeteRAG* shows limitations in regression tasks which pointing to the need for a meta-learning component to guide its iterative process in future works. Overall, our solution establishes a clear trade-off by reducing human effort while consuming little computation to achieve near-human performance.

VIII. Data Availability

Benchmarks and results are available in this repository [51].

IX. Acknowledgment

This work was supported by the NSF grants #CNS-2349663 and #OAC-2528533. This work used Indiana JetStream2 GPU at Indiana University through allocation NAIRR250048 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by the NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296. This work was partially supported by the CyberCorps Scholarship for Service program at California State University, Sacramento. Any opinions, findings, and conclusions or recommendations expressed in this work are those of the author(s) and do not necessarily reflect the views of the NSF.

References

[1] Kaggle, "Kaggle website," https://www.kaggle.com, 2025, accessed: August 12, 2025.

[2] A. Tolkachev, I. Sirazitdinov, M. Kholiavchenko, T. Mustafaev, and B. Ibragimov, "Deep learning for diagnosis and segmentation of pneumothorax: the results on the kaggle competition and validation against radiologists," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 5, pp. 1660–1672, 2020.

[3] I. Vallés-Pérez, E. Soria-Olivas, M. Martínez-Sober, A. J. Serrano-López, J. Gómez-Sanchís, and F. Mateo, "Approaching sales forecasting using recurrent neural networks and transformers," *Expert Systems with Applications*, vol. 201, p. 116993, 2022.

[4] G. Sambasivam and G. D. Opiyo, "A predictive machine learning application in agriculture: Cassava disease detection and classification with imbalanced dataset using convolutional neural networks," *Egyptian Informatics Journal*, vol. 22, no. 1, pp. 27–34, 2021.

[5] K. S. Babu and M. M. Kodabagi, "An efficient framework for predicting future retail sales using ensemble dnn-bilstm technique," *SN Computer Science*, vol. 5, no. 1, p. 150, 2024.

[6] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[7] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.

[8] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.

[9] F. Batole, D. OBrien, T. N. Nguyen, R. Dyer, and H. Rajan, "An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, no. 16. IEEE Computer Society, 2025, pp. 637–637.

[10] T. Coignion, C. Quinton, and R. Rouvoy, "A performance study of LLM-generated code on leetcode," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, no. 17, 2024, pp. 79–89.

[11] A. Terbuch, "LSTM Hyperparameter Optimization: Impact of the Selection of Hyperparameters on Machine Learning Performance when Applied to Time Series in Physical Systems," Master's thesis, Graz University of Technology, 2021.

[12] W. Reade and A. Chow, "Tabular Playground Series - May 2022," https://kaggle.com/competitions/tabular-playground-series-may-2022, 2022, kaggle.

[13] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "Clarifygpt: A framework for enhancing LLM-based code generation via requirements clarification," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2332–2354, 2024.

[14] H. Koziolek, S. Grüner, R. Hark, V. Ashiwal, S. Linsbauer, and N. Eskandani, "LLM-based and Retrieval-Augmented Control Code Generation," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, 2024, pp. 22–29.

[15] Kaggle, "Kaggle Playground Competitions," https://www.kaggle.com/competitions?hostSegmentIdFilter=8, 2025, accessed: August 17, 2025.

[16] ——, "API Documentation," https://www.kaggle.com/docs/api, 2025, accessed: August 12, 2025.

[17] Y.-S. Chuang, R. Dangovski, H. Luo, Y. Zhang, S. Chang, M. Soljačić, S.-W. Li, S. Yih, Y. Kim, and J. Glass, "DiffCSE: Difference-based Contrastive Learning for Sentence Embeddings," *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, no. 25, 2022.

[18] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 26, pp. 535–547, 2021.

[19] N. Shawki, R. Rodriguez Nunez, I. Obeid, and J. Picone, "On Automating Hyperparameter Optimization for Deep Learning Applications," in *Proc. IEEE Signal Processing in Medicine and Biology Symposium (SPMB)*, no. 27, Philadelphia, PA, USA, 2021, pp. 1–7.

[20] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, L. Invernizzi *et al.*, "KerasTuner," https://github.com/keras-team/keras-tuner, 2019.

[21] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 29, pp. 281–305, 2012.

[22] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[23] L. Hertel, J. Collado, P. Sadowski, J. Ott, and P. Baldi, "Sherpa: Robust Hyperparameter Optimization for Machine Learning," *SoftwareX*, vol. 12, no. 31, p. 100591, 2020.

[24] A. Taulananda, "Analyzing Efficiency-Based Hyperparameter Tuning Optimization Methods on LSTMs for Generative ARIMA Models," *International Journal of High School Research*, vol. 6, no. 32, pp. 10–18, 2024.

[25] "CompeteRAG Prompts," https://github.com/Laboratory-software-Innovation/CompeteRAG/blob/main/Experiments-o4-mini/Prompts.pdf, 2025, accessed: 29-08-2025.

[26] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-sklearn 2.0: Hands-free automl via meta-learning," *Journal of Machine Learning Research*, vol. 23, no. 2, pp. 1–61, 2022.

[27] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2013, pp. 847–855.

[28] H. Jin, Q. Song, and X. Hu, "Auto-Keras: An efficient neural architecture search system," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, no. 4, 2019, pp. 1946–1956.

[29] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola, "AutoGluon-Tabular: Robust and accurate automl for structured data," *arXiv preprint arXiv:2003.06505*, no. 5, 2020.

[30] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, no. 6, 2016.

[31] G. Nguyen, M. J. Islam, R. Pan, and H. Rajan, "Manas: Mining software repositories to assist automl," in *Proceedings of the 44th International Conference on Software Engineering*, no. 7, 2022, pp. 1368–1380.

[32] P. Trirat, W. Jeong, and S. J. Hwang, "AutoML-Agent: A multi-agent LLM framework for full-pipeline automl," *arXiv preprint arXiv:2410.02958*, no. 8, 2024.

[33] D. Luo, C. Feng, Y. Nong, and Y. Shen, "Autom3l: An automated multimodal machine learning framework with large language models," in *Proceedings of the 32nd ACM International Conference on Multimedia*, no. 9, 2024, pp. 8586–8594.

[34] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, no. 10, 2021.

[35] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, no. 11, 2023.

[36] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, no. 12, 2022.

[37] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, no. 13, 2021.

[38] R. Tóth, T. Bisztray, and L. Erdődi, "LLMs in Web Development: Evaluating LLM-Generated PHP Code Unveiling Vulnerabilities and Limitations," in *International Conference on Computer Safety, Reliability, and Security*, no. 18. Springer, 2024, pp. 425–437.

[39] M. A. Shehab, M. Wardat, S. Omari, and Y. Jararweh, "Evaluating large language models for code generation: Assessing accuracy, quality, and performance," in *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*. IEEE, 2024, pp. 407–416.

[40] W. Wang, Y. Wang, S. Joty, and S. C. Hoi, "RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, no. 14, 2023, pp. 146–158.

[41] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, no. 15. IEEE, 2023, pp. 1469–1481.

[42] S. Omari, K. Basnet, and M. Wardat, "Investigating large language models capabilities for automatic code repair in python," *Cluster Computing*, vol. 27, no. 8, pp. 10 717–10 731, 2024.

[43] E. Mansur, J. Chen, M. A. Raza, and M. Wardat, "RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts," in *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024, pp. 7491–7496.

[44] Z. Xue, L. Li, S. Tian, X. Chen, P. Li, L. Chen, T. Jiang, and M. Zhang, "LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, no. 19, 2024, pp. 1643–1655.

[45] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for LLM-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, no. 20, 2024, pp. 572–576.

[46] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," *arXiv preprint arXiv:2303.12570*, no. 21, 2023.

[47] C. Paduraru, A. Stefanescu, and A. Jianu, "Unit Test Generation using Large Language Models for Unity Game Development," in *Proceedings of the 1st ACM International Workshop on Foundations of Applied Software Engineering for Games*, no. 22, 2024, pp. 7–13.

[48] X. V. Lin, X. Chen, M. Chen, W. Shi, M. Lomeli, R. James, P. Rodriguez, J. Kahn, G. Szilvasy, M. Lewis *et al.*, "Ra-dit: Retrieval-augmented dual instruction tuning," in *The Twelfth International Conference on Learning Representations*, no. 23, 2023.

[49] M. Mukherjee and V. J. Hellendoorn, "Sosecure: Safer code generation with rag and stackoverflow discussions," *arXiv preprint arXiv:2503.13654*, no. 24, 2025.

[50] C. Xie, M. Jiao, X. Gu, and B. Shen, "Empowering AI to Generate Better AI Code: Guided Generation of Deep Learning Projects with LLMs," *arXiv preprint arXiv:2504.15080*, 2025.

[51] "Github Repository: CompeteRAG," https://github.com/Laboratory-software-Innovation/CompeteRAG, 2025, accessed 29-08-2025.